



机器学习实验一：感知机模型

姓名：冯怡然

学号：2411434

班级：人工智能特色班

日期：2026 年 4 月 29 日

南开大学 · 机器学习实验

1 实验目的

- 深入理解感知机（Perceptron）模型的基本理论、几何意义与数学表达式。
- 掌握感知机学习算法（PLA）及其随机梯度下降法（SGD）的实现逻辑。
- 通过编程，探究分类超平面的生成与演变过程。
- 分析超参数（如学习率、初始化权重）及数据分布（线性可分与不可分）对模型收敛性与泛化能力的影响。

2 实验内容

- 基础任务：**使用给定二维高斯分布的训练数据（中心分别为 $(1, 1)$ 和 $(3, 4)$ ）训练感知机模型，并在二维平面画出分类线。
- 测试与评估：**使用训练好的模型对测试数据进行分类，并报告分类准确率。
- 附加题 5 道**

3 实验原理

3.1 模型定义

感知机由 $y = f(x) = \text{sign}(w \cdot x + b)$ 定义，其中 x 为样本特征向量， $y \in \{-1, +1\}$ 为类别标签。 w 为权重向量， b 为偏置。

3.2 损失函数

通过最小化误分类点到超平面的总距离来训练模型，目标函数为：

$$L(w, b) = - \sum_{x_i \in M} y_i (w \cdot x_i + b) \quad (1)$$

其中 M 是误分类点的集合。

3.3 训练算法

使用随机梯度下降法（SGD）更新参数：

- 若样本 (x_i, y_i) 被误分类，即 $y_i(w \cdot x_i + b) \leq 0$ ：

$$w \leftarrow w + \eta y_i x_i \quad (2)$$

$$b \leftarrow b + \eta y_i \quad (3)$$

- 重复迭代直到 M 为空集或达到最大迭代次数。

4 基础题：感知机训练逻辑

4.1 代码实现

```
1 % 数据生成
2 n = 100; % 样本量大小
3 center1 = [1,1]; % 第一类数据中心
4 center2 = [3,4]; % 第二类数据中心
5 X = zeros(2*n,2); % 2n *
    2的数据矩阵，每一行表示一个数据点，第一列表示x轴坐标，第二列表示y轴坐标
6 Y = zeros(2*n,1); % 类别标签
7 X(1:n,:) = ones(n,1)*center1 + randn(n,2); %生成数据：中心点+高斯噪声
8 X(n+1:2*n,:) = ones(n,1)*center2 + randn(n,2);
    %矩阵X的前n行表示类别1中数据，后n行表示类别2中数据
9 Y(1:n) = 1;
10 Y(n+1:2*n) = -1; % 第一类数据标签为1，第二类为-1
11 figure(1)
12 set(gcf,'Position',[1,1,700,600], 'color','w')
13 set(gca,'FontSize',18)
14 plot(X(1:n,1),X(1:n,2),'ro','LineWidth',1,'MarkerSize',10); %
    画第一类数据点
15 hold on;
16 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10); %
    画第二类数据点
17 hold on;
18 xlabel('x axis');
19 ylabel('y axis');
20 legend('class 1','class 2');
21 % 感知机模型，学生实现，训练感知机模型的参数(w,b)
22 w = zeros(2,1);
23 b = zeros(1); % 感知机模型  $y = x*w + b$ 
24 learningRate=0.04;
25 maxIter=1000;
26 for iter=1:maxIter
27     hasError=false;
28     for i=1:2*n
29         if Y(i)*(X(i,:)*w+b)<=0
30             w=w+learningRate*Y(i)*X(i,:);
31             b=b+learningRate*Y(i);
32             hasError=true;
33         end
34     end
35     if ~hasError
36         fprintf('已收敛，迭代次数为: %d\n',iter)
37         break;
38     end
39 end
40 % 分类器可视图，即画出  $x*w + b = 0$  的图像
41 x1 = -2 : 0.00001 : 7;
42 y1 = ( -b * ones(1,length(x1)) - w(1) * x1 )/w(2);
```

```

43 % 分类界面,length()表示向量长度
44 % x1为分类界面横轴,y1为纵轴
45 figure(3)
46 set(gcf,'Position',[1,1,700,600], 'color','w')
47 set(gca,'FontSize',18)
48 plot(X(1:n,1),X(1:n,2),'ro','LineWidth',1,'MarkerSize',10); %
    画第一类数据点
49 hold on;
50 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10); %
    画第二类数据点
51 hold on;
52 plot(x1,y1,'k','LineWidth',1,'MarkerSize',10); % 画分类界面
53 xlabel('x axis');
54 ylabel('y axis');
55 legend('class 1','class 2','classification surface');
56
57 %生成测试数据:与训练数据同分布
58 m = 10; % 测试样本量大小
59 Xt = zeros(2*m,2); % 2n *
    2的数据矩阵, 每一行表示一个数据点, 第一列表示x轴坐标, 第二列表示y轴坐标
60 Yt = zeros(2*m,1); % 类别标签
61 Xt(1:m,:) = ones(m,1)*center1 + randn(m,2);
62 Xt(m+1:2*m,:) = ones(m,1)*center2 + randn(m,2);
    %矩阵X的前n行表示类别1中数据, 后n行表示类别2中数据
63 Yt(1:m) = 1;
64 Yt(m+1:2*m) = -1; % 第一类数据标签为1, 第二类为-1
65 figure(4)
66 set(gcf,'Position',[1,1,700,600], 'color','w')
67 set(gca,'FontSize',18)
68 plot(X(1:n,1),X(1:n,2),'ro','LineWidth',1,'MarkerSize',10); %
    画第一类数据点
69 hold on;
70 plot(X(n+1:2*n,1),X(n+1:2*n,2),'b*','LineWidth',1,'MarkerSize',10); %
    画第二类数据点
71 hold on;
72 plot(Xt(1:m,1),Xt(1:m,2),'go','LineWidth',1,'MarkerSize',10); %
    画第一类数据点
73 hold on;
74 plot(Xt(m+1:2*m,1),Xt(m+1:2*m,2),'g*','LineWidth',1,'MarkerSize',10); %
    画第二类数据点
75 hold on;
76 plot(x1,y1,'k','LineWidth',1,'MarkerSize',10); %
    画分类界面
77 xlabel('x axis');
78 xlabel('x axis');
79 ylabel('y axis');
80 legend('class 1: train','class 2: train','class 1: test','class 2:
    test','classification surface');
81 %学生实现: 给出模型的预测输出, 并与测试数据的真实输出比较, 计算错误率
82 % 预测并计算错误率

```

```

83 Y_pred = zeros(2*m, 1); % 存储预测结果
84 errors = 0; % 错误计数器
85 for i = 1:2*m % 使用训练好的 w, b 进行预测
86     if (Xt(i,:) * w + b) >= 0
87         Y_pred(i) = 1;
88     else
89         Y_pred(i) = -1;
90     end
91     % 检查预测是否与真实标签 Yt 一致
92     if Y_pred(i) ~= Yt(i)
93         errors = errors + 1;
94     end
95 end
96 error_rate = errors / (2*m); % 计算错误率
97 fprintf('测试样本总数: %d, 错误个数: %d, 错误率: %.2f%%\n', 2*m, errors,
    error_rate * 100);

```

Perceptron Training

4.2 运行结果

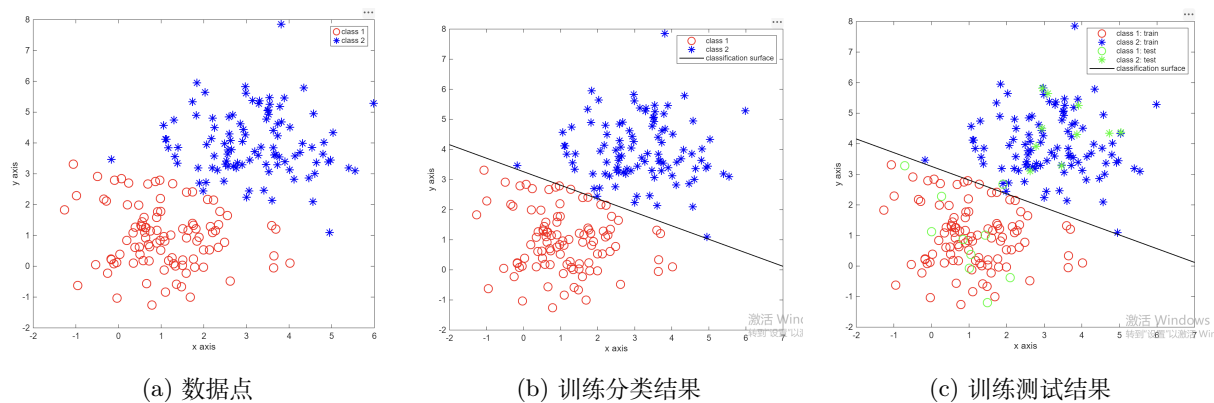


图 1: 感知机实验运行结果展示

1. 已收敛, 迭代次数为: 73
2. 测试样本总数: 20, 错误个数: 1, 错误率: 5.00 %

5 附加题

5.1 改变步长 (learning rate)

1. 在 $w = [0, 0], b = 0$ 时通过改变学习率得到以下结果

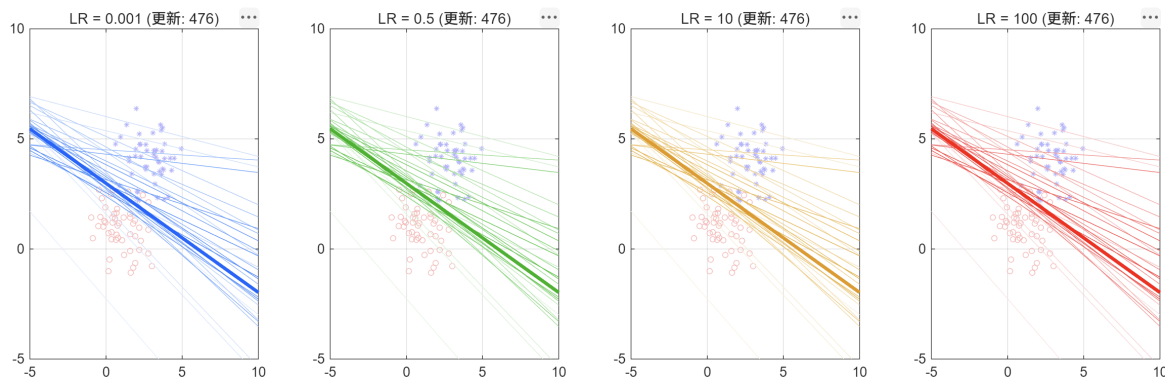


图 2: 零初始条件测试数据结果

在零初始条件下, 学习率的改变不影响迭代次数和最终结果

2. 在 $w = [0.7; -0.5], b = 0.1$ 时通过改变学习率得到以下结果

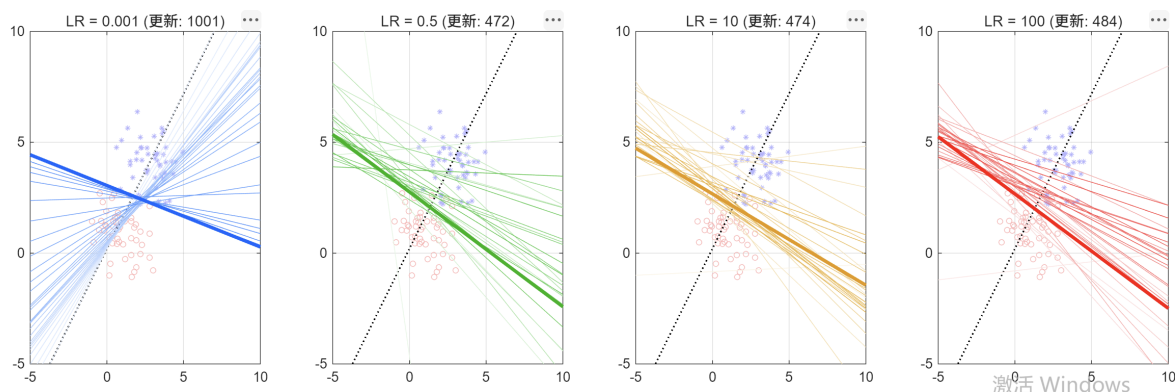


图 3: 非零初始条件测试数据结果

在非零初始条件下, 随着学习率的提升, 收敛速度会变快。
但是当学习率过大的时候, 会产生震荡, 影响收敛速度。

3. 原因解释在感知机中, 更新公式是:

$$w_{new} = w_{old} + \eta \cdot y_i \cdot x_i$$

$$b_{new} = b_{old} + \eta \cdot y_i$$

如果从 $w = 0, b = 0$ 开始, 那么学习率 η 仅仅是一个缩放因子。如果 $\eta = 10$, 更新后的权重可能是 $[10, 20]$ 。如果 $\eta = 100$, 更新后的权重可能是 $[100, 200]$ 。虽然数值不同, 但它们指向的方向完全一致。对于决策边界 $w \cdot x + b = 0$ 来说, 方向决定了直线的位置。

5.2 去掉收敛条件

5.2.1 有收敛条件

含有收敛条件时，当数据线性可分

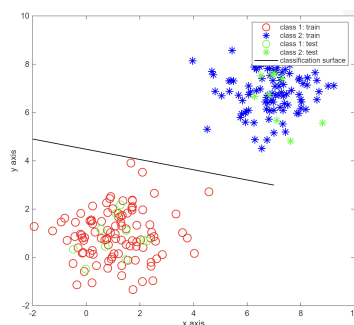


图 4: 收敛条件测试数据结果

已收敛，迭代次数为：24

【普通算法 A】实际迭代次数: 24, 耗时: 0.0011 秒

5.2.2 无收敛条件

不含有收敛条件时得到

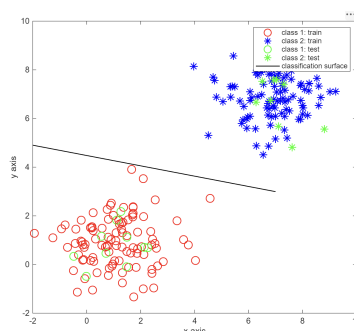


图 5: 无收敛条件测试数据结果

【普通算法 A】实际迭代次数: 1000, 耗时: 0.0254 秒

5.2.3 原理解释

在感知机算法中，原本的更新规则是“仅当分类错误时才更新”：

$$y_i(w^T x_i + b) \leq 0$$

如果去掉这个收敛触发条件，会发生以下情况：

1. 失去稳定性：即使 w 已经将两类点完美分开，强制更新也会让分类平面向某个数据点方向偏移，从而破坏已有的分类效果，但这一点在实验中并不明显。
2. 运行时间变长：这一点可以从实验结果中看出。

从这个实验的结果来看，收敛条件对于减少计算复杂度有很大作用。

5.3 线性不可分

对于线性不可分的情况，循环不能终止，因为 $Y(i) * (X(i,:) * w + b) \leq 0$ 的条件永远存在，因为不会终止，只会一直迭代下去，直到达到最大迭代次数。运用 pocket 算法可以避免迭代带来的误差，下图中蓝色虚线代表 pocket 算法运行结果。

5.3.1 pocket 算法代码实现

```
1 tic; iter_B = 0;
2 for iter = 1:maxIter
3     has_update = false;
4     for i = 1:2*n
5         if Y(i)*(X(i,:)*wb + bb) <= 0
6             wb = wb + learningRate * Y(i) * X(i,:); % 更新当前参数
7             bb = bb + learningRate * Y(i);
8             has_update = true;
9             preds = (X * wb + bb); % 评估新参数性能
10            current_errors = sum((Y .* preds) <= 0);
11            if current_errors < best_errors
12                wb_pocket = wb;
13                bb_pocket = bb;
14                best_errors = current_errors;
15                best_iter = iter; end
16            break; end
17        end
18        iter_B = iter;
19        if ~has_update || best_errors == 0 break; end
20    end
21    wb = wb_pocket; bb = bb_pocket; time_B = toc;
```

Perceptron Training

5.3.2 运行结果

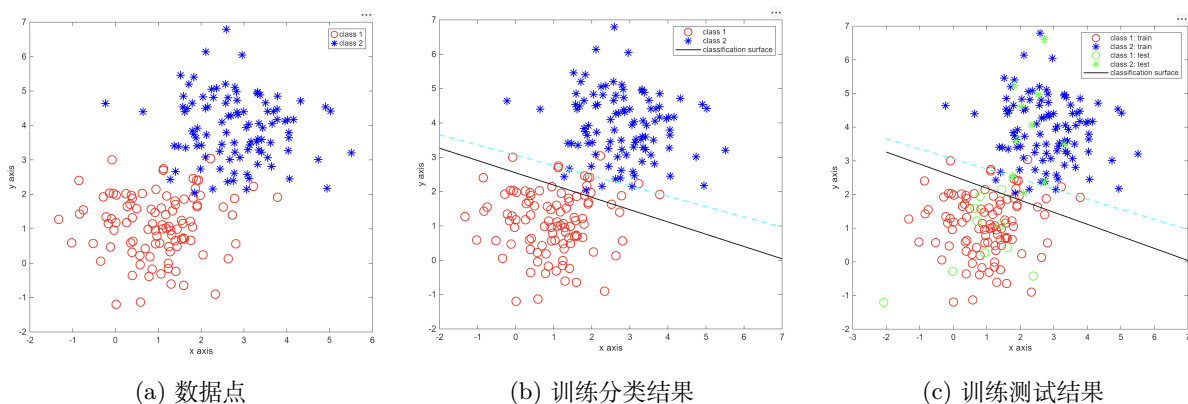


图 6: 算法对比运行结果展示


```
>> perception_student
【普通算法A】实际迭代次数: 1000, 耗时: 0.0162 秒
【Pocket算法B】最佳迭代次数: 462, 训练集最小错误数: 6, 耗时: 0.0082 秒
测试样本总数: 20, 错误个数: 1, 错误率: 5.00%
测试样本总数: 20, 错误点数: 1, 错误率: 5.00%
```

图 7: 测试数据结果

5.4 高维数据

5.4.1 3 维实现代码

```
1 %3D 高维数据生成
2 rng(0);
3 n = 100; % 样本量大小
4 center1 = [1, 1, 1]; % 第一类数据中心 (3D)
5 center2 = [4, 4, 4]; % 第二类数据中心 (3D)
6 X = zeros(2*n, 3); % 2n * 3 的数据矩阵
7 Y = zeros(2*n, 1); % 类别标签
8 % 生成数据: 中心点 + 高斯噪声
9 X(1:n,:) = ones(n,1)*center1 + randn(n,3);
10 X(n+1:2*n,:) = ones(n,1)*center2 + randn(n,3);
11 Y(1:n) = 1; Y(n+1:2*n) = -1;
12 figure(1)
13 set(gcf, 'Position', [100, 100, 800, 700], 'color', 'w')
14 plot3(X(1:n,1), X(1:n,2), X(1:n,3), 'ro', 'LineWidth', 1, 'MarkerSize', 8); hold
    on;
15 plot3(X(n+1:2*n,1), X(n+1:2*n,2), X(n+1:2*n,3), 'b*', 'LineWidth', 1,
    'MarkerSize', 8);
16 grid on; view(3); % 开启3D视图
17 xlabel('X轴'); ylabel('Y轴'); zlabel('Z轴');
18 legend('类别 1', '类别 2');
19 title('3D 原始数据分布');
20 % 3D 感知机模型训练
21 w = zeros(3,1); b = 0; % 3D权重向量 [w1; w2; w3]
22 learningRate = 0.1;
23 maxIter = 5000;
24 update_count = 0;
25 tic;
26 for iter = 1:maxIter
27     hasError = false;
28     for i = 1:2*n
29         if Y(i) * (X(i,:) * w + b) <= 0
30             w = w + learningRate * Y(i) * X(i,:);
31             b = b + learningRate * Y(i);
32             hasError = true;
33             update_count = update_count + 1;
34         end
35     end
36     if ~hasError
37         fprintf('已收敛, 迭代步数: %d, 更新次数: %d\n', iter, update_count);
```

```

38         break;
39     end
40 end
41 training_time = toc;
42 figure(2)
43 set(gcf, 'Position', [950, 100, 800, 700], 'color', 'w')
44 plot3(X(1:n,1), X(1:n,2), X(1:n,3), 'ro', 'MarkerSize', 6); hold on;
45 plot3(X(n+1:2*n,1), X(n+1:2*n,2), X(n+1:2*n,3), 'b*', 'MarkerSize', 6);
46 % 创建网格来绘制平面
47 [x_mesh, y_mesh] = meshgrid(-2:0.5:7, -2:0.5:7);
48 z_mesh = (-b - w(1)*x_mesh - w(2)*y_mesh) / w(3);
49 surf(x_mesh, y_mesh, z_mesh, 'FaceAlpha', 0.3, 'EdgeColor', 'none', 'FaceColor',
    [0.5 0.5 0.5]);
50 colormap jet;
51 grid on; view(3);
52 xlabel('X轴'); ylabel('Y轴'); zlabel('Z轴');
53 title(['3D感知机分类结果 (耗时: ', num2str(training_time, '%.4f'), 's)']);
54 legend('类别 1', '类别 2', '分类超平面');
55 % 测试阶段
56 m = 20;
57 Xt = [ones(m,1)*center1 + randn(m,3); ones(m,1)*center2 + randn(m,3)];
58 Yt = [ones(m,1); -ones(m,1)];
59 Y_pred = sign(Xt * w + b);
60 errors = sum(Y_pred ~= Yt);
61 fprintf('测试样本总数: %d, 错误个数: %d, 错误率: %.2f%%\n', 2*m, errors,
    (errors/(2*m))*100);

```

Extra Tasks

5.4.2 运行结果

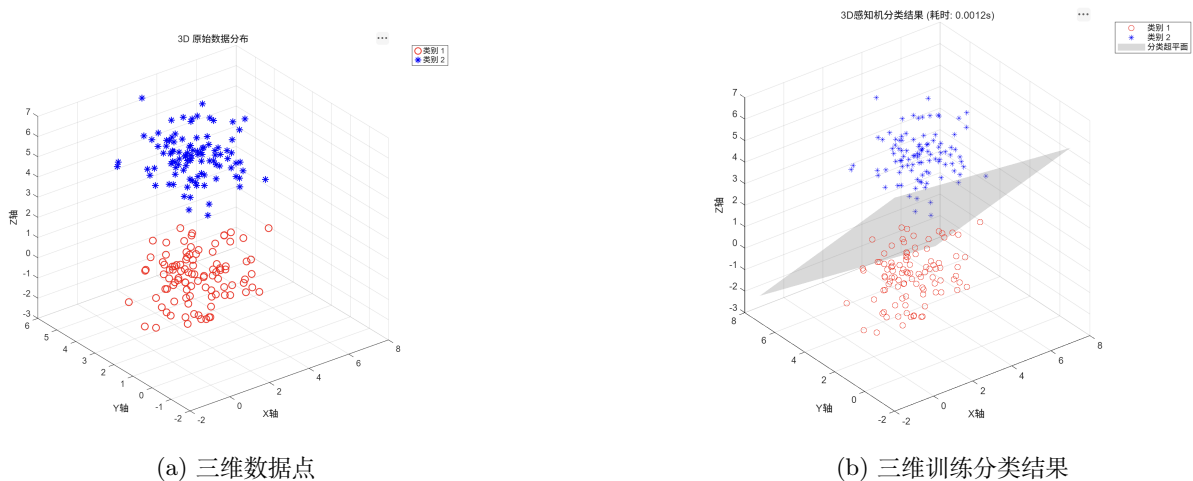


图 8: 三维感知机实验运行结果展示

5.4.3 100 维感知机分类运行结果

相对应代码在 add3.m 里，由于篇幅限制不全部放在这里了。

测试数据如下

```
1 rng(0);
2 n = 100;           % 训练样本量（每类100个，共200个）
3 m = 20;           % 测试样本量（每类20个，共40个）
4 dims = 100;       % 数据维度：100维
5 center1 = ones(1, dims) * 1;
6 center2 = ones(1, dims) * 3;
7 % 生成训练数据
8 X = [ones(n,1)*center1 + randn(n, dims);
9       ones(n,1)*center2 + randn(n, dims)];
10 Y = [ones(n,1); -ones(n,1)];
11 % 生成测试数据
12 Xt = [ones(m,1)*center1 + randn(m, dims);
13        ones(m,1)*center2 + randn(m, dims)];
14 Yt = [ones(m,1); -ones(m,1)];
```

Extra Tasks

通过 PCA 的方法实现数据降维，得到结果如下，其中错误分类点用梅红色圆圈标出

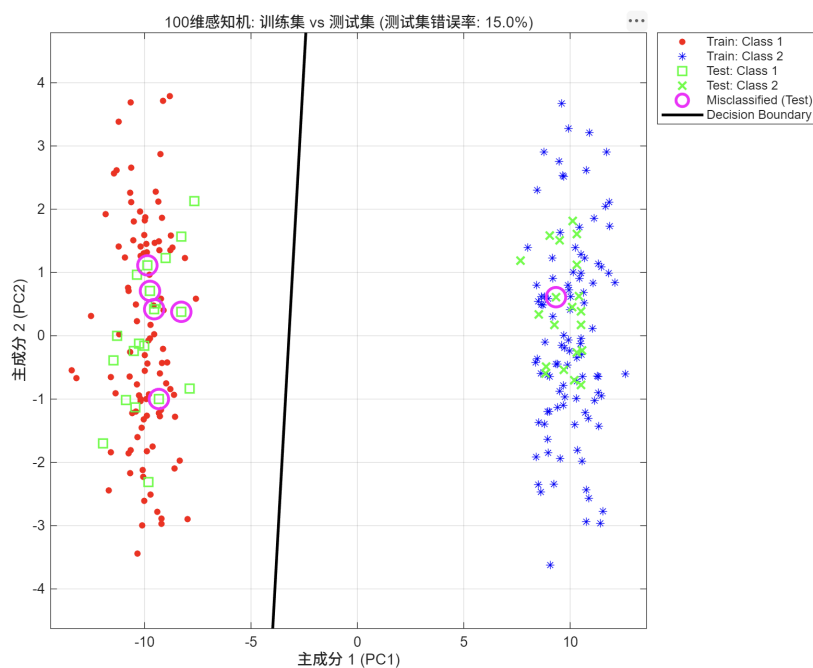


图 9: 三维感知机实验运行结果展示

由此可见，感知机对于高纬度的数据具有分类能力。但具体是否符合维度更高，分类效果越差，在这个实验里仍有待测试。

5.5 改变数据中心距离

5.5.1 增大数据中心距离

将数据集进行如下修改

```
1 % 数据中心设置
2 center1 = [1, 1];
3 center2 = [40, 20];
4 % 生成数据
5 X = [center1 + randn(n,2); center2 + randn(n,2)];
6 Y = [ones(n,1); -ones(n,1)];
```

Extra Tasks

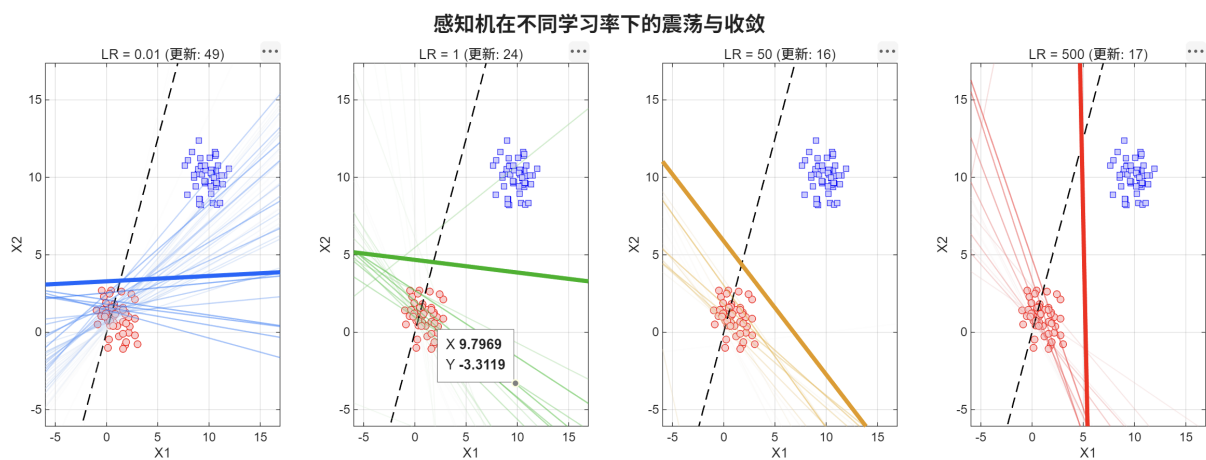


图 10: 增大数据中心距离测试数据结果

可以看到一定范围内增大学习率，对于数据中心距离大的情况，学习效果更好。

5.5.2 减小数据中心距离

将数据集进行如下修改

```
1 % 数据中心设置
2 center1 = [1, 1];
3 center2 = [0.5, 0.5];
4 % 生成数据
5 X = [center1 + 0.1*randn(n,2); center2 + 0.1*randn(n,2)];
6 Y = [ones(n,1); -ones(n,1)];
```

Extra Tasks

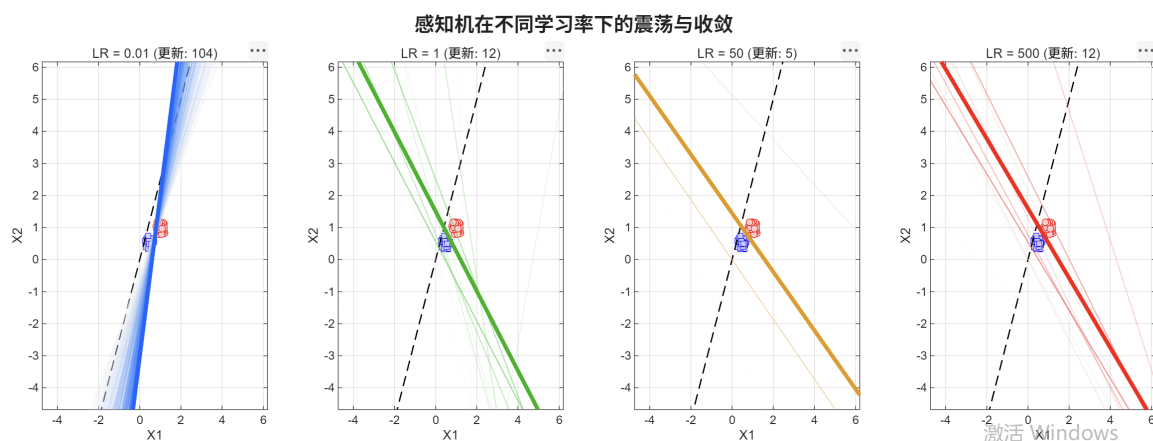


图 11: 减小数据中心距离测试数据结果

可以看出感知机得到的分类结果，在数据中心距离较大的时候并不一定时最优结果。无论距离远近，感知机分类结果都和学习率有很大关系。对于不同数据中心距离的情况，最佳的学习率有所不同。

6 实验总结收获

结合理论与实验结果看到，感知机算法可以进行简单的分类，但由于它是以误差函数为中心的算法，在训练的时候它只能保证误差最小，但无法达到最优分类的结果，因此例如 SVM 算法具有超越感知机算法局限的内涵。

通过这次实验的大量调试，我对于感知机的算法内容和学习效果有了更加深刻的理解。在修改代码和调试参数的过程中，我也深感机器学习是一门在大量实验基础下的学科。我的实验结果印证了大部分数学理论的推导，而且补充了我对于理论理解的不足，从中我获得了极大的成就感。这也让我更加喜欢机器学习的内容，并且愿意去进一步探索。